

Benchmark Report: nbody

Five bodies, twenty thousand steps, and a sixth of the runtime spent computing the number zero

Matan Cohen · Yuval Kogan · HWSW Final Project, Technion · github.com/matanco64/HWSW_Project

A probe is falling toward Neptune, and somebody has to know where it will be in six hours. That is this benchmark: integrate the Sun and the four gas giants forward under Newtonian gravity, twenty thousand steps at a time. It is thirty lines of arithmetic with no libraries, no I/O, and no data structure more exotic than a list of floats — which makes it the cleanest possible place to ask a question with a surprising answer: **when CPython runs this, what is it actually doing?** Almost none of it is gravity. Roughly a sixth of the runtime goes into converting the constant 0 into a C array index, twenty thousand times over, and that observation is what the optimization in §3 and the accelerator in §5 are both built on.

1. Overview

nbody comes from the Computer Language Benchmarks Game: a **symplectic-Euler integration** of the Sun plus Jupiter, Saturn, Uranus and Neptune under Newtonian gravity. Five bodies means **ten interacting pairs**, computed once at import and reused for the whole run. Each pyperf iteration runs `report_energy()`, `advance(dt=0.01, n=20000)`, then `report_energy()` again.

- **Libraries:** `stdlib` only — the stock version does not even import `math`. Nothing is vectorized, nothing is compiled, there is no I/O in the timed region.
- **Data structures:** a list of `[position, velocity, mass]` triples, where position and velocity are 3-element lists of floats; `PAIRS` is a list of 2-tuples of those triples. Total mutable state is **35 floats**.
- **Correctness oracle:** total system energy. A symplectic integrator conserves it to $O(dt)$, so any change to the physics shows up immediately.

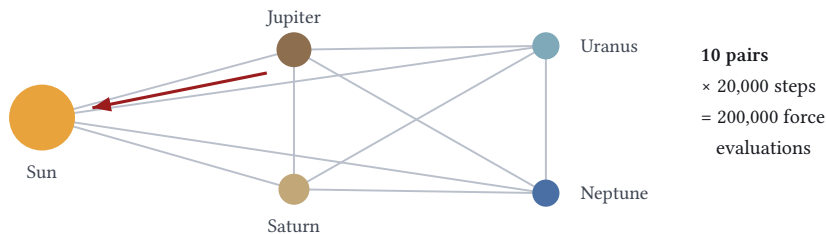


Figure 1: The entire workload. Because $N = 5$, the pair list is **fixed for the whole run** — the fact §3 exploits.

2. Initial analysis

Measurement setup. Course QEMU VM: Ubuntu 22.04, CPython 3.10.12, pyperformance 1.14.0, KVM on a Technion host. Timings are `pyperformance run - - rigorous` (40 processes / 120 values) on the release `python3`. Profiles are recorded separately with `python3-dbg` for symbols and are **never** quoted as timings — the debug build’s assertions and allocator hooks inflate interpreter-internal frames roughly 2.5–3×. Two guest quirks worth recording: the `cycles` PMU event records **zero** samples under KVM, so sampling must use `-e cpu-clock`; and `perf stat` silently returns zeros past four events, so counters are taken in two passes.

Stock **nbody** measures **231 ms ± 3 ms**, reproduced to within 1 ms across runs a week apart — an unusually stable benchmark. The flame graph is a single `advance()` plateau over `_PyEval_EvalFrameDefault`, about 95% of samples. The interesting part is underneath it:

Cost group	Self	Symbols
List subscript machinery	~15%	<code>list_ass_item</code> 2.83, <code>PyNumber_AsSsize_t</code> 2.12, <code>_PyNumber_Index</code> 1.87, <code>PyObject_GetItem</code> 1.65, <code>list_item</code> 1.65, <code>PyObject_SetItem</code> 1.47, <code>PyLong_AsSsize_t</code> 1.43, <code>list_ass_subscript</code> 1.33
Generic binary-op dispatch	~8%	<code>binary_op1</code> 5.75, <code>_Py_CheckSlotResult</code> 2.22
Float boxing	large	<code>PyFloat_FromDouble</code> / <code>float_dealloc</code> churn
Actual arithmetic	small	<code>libm pow</code> for <code>d**-1.5</code> , plus the adds and multiplies

On CPython 3.10 every `v1[0] -= dx * b2m` is a full `PyObject_GetItem` → `_PyNumber_Index` → `PyLong_AsSsize_t` → `list_item` round trip, and the reverse for the store. **None of that is physics.** CPython 3.11 later added `BINARY_SUBSCR_LIST_INT` specialization to make exactly this cheap; 3.10 — the version the course VM runs — has none of it.

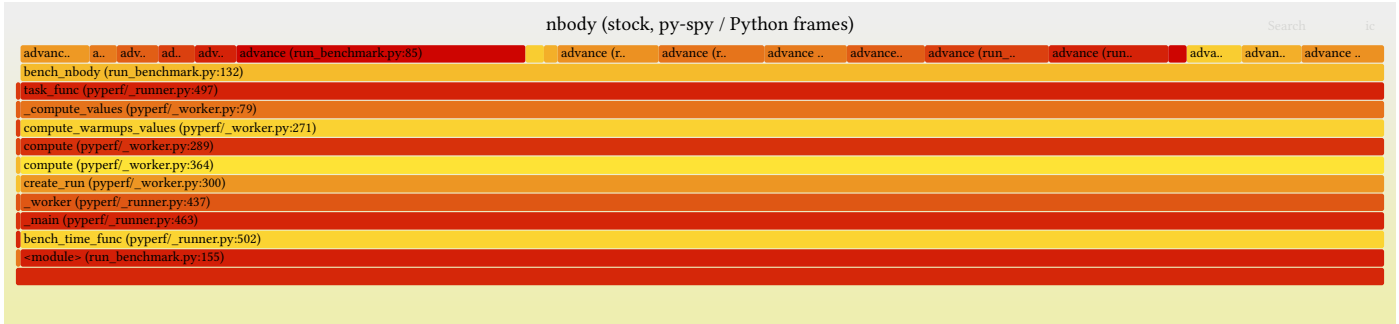


Figure 2: **Before, Python frames** (py-spy). CPython 3.10 has no `-X perf` trampoline, so `perf` cannot name Python functions; py-spy samples the interpreter’s own frame stack. `advance` is the single wide plateau under `bench_nbody`. 5 of 303 samples (1.6%) were taken while CPython was still importing modules and are excluded, which is what takes the figure from 36 rows to 12; see [the appendix](#).

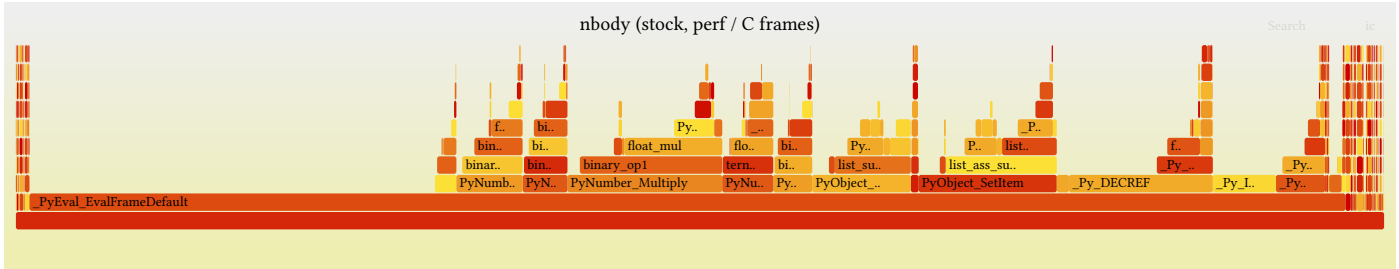


Figure 3: **Before, C frames** (perf, DWARF unwinding, python3-dbg). 178 rows reduced to 9 by two cuts that are *not* equally innocent. The 97 leading frames shared by $\geq 95\%$ of samples — interpreter start-up and the pyperf harness — are elided: constant context, no information, and every surviving width is untouched. Stacks are then capped at depth 9, which merges away the tips of 4.5% of samples; that one does discard detail. The leaves that remain are the cost groups tabulated above. Uncut version: [results/flame_nbody_stock_full.svg](#), and [the appendix](#) for the method.

A note on how these were recorded. Our first captures used `perf record -g` (frame-pointer unwinding) and were **unusable**: 1,877 [unknown] frames and call chains full of return addresses like `0xfdfdfdfdfdfdf00` — the `Py_DEBUG` freed-memory fill byte. Ubuntu’s `python3-dbg` is built without frame pointers, so the unwinder was walking into freed stack memory. `--call-graph dwarf`, 16384 fixes it (0 such addresses, 105 unknowns) at the cost of $\sim 100\times$ larger captures and ~ 20 minutes per profile. The flat self-percentages above were always valid; only the call graph was affected.

3. Optimizations

3a. What shipped: compile the schedule away

The inner loop rediscovers, twenty thousand times, a schedule that is **constant for the entire run**: which bodies interact, in what order, with which masses. Only the 30 coordinate floats change. So we compute that schedule once, at import time and outside the timed region, and emit straight-line source for `advance(dt, n)` in which every coordinate is a Python **local** (`LOAD_FAST`) and every mass is a **literal** (`LOAD_CONST`). State is read out of the body lists before the step loop and written back after, so `report_energy()` observes the same objects. This is **partial evaluation** — the technique `dataclasses` and Django’s template compiler use.

Per integration step	Stock	Generated
Bytecodes executed	1484.5	850.6
List subscripts (<code>BINARY_SUBSCR/STORE_SUBSCR</code>)	150	4

Removing 146 of 150 subscripts per step attacks precisely the $\sim 15\%$ tower in §2. The bytecode counts come from `sys.monitoring` and are a **version-independent structural metric**: unlike wall clock, they do not depend on which CPython specializes what.

The output is bit-for-bit identical. All 35 state floats compare `==` against stock after 20,000 steps, and so does `report_energy()`: $\max |\Delta| = 0.0$ — an equality, not a tolerance. We kept the stock `dt * (dsq ** -1.5)` rather than the cheaper `dt / (dsq * sqrt(dsq))` to preserve that claim, and measurement justified the choice independently: the `sqrt` variant was not distinguishable from zero on any interpreter tested, and on one run came out slower. The emitter is **generic in N** — hand it nine bodies and it emits a nine-body kernel — so this specializes the given workload rather than hard-coding an answer.

3b. Three anti-results

- **Fast Multipole / Barnes–Hut is a $3.4\times$ slowdown at $N = 5$.** We built a real octree Barnes–Hut (monopole, $\theta = 0.5$, tree rebuilt every step, validated by driving $\theta \rightarrow 0$ to reproduce direct summation to $1.2e-16$) and measured the crossover: BH/direct is **3.39** at $N = 5$, 1.58 at $N = 100$, and only drops below 1.0 near $N \approx 300\text{--}400$ — this benchmark runs $70\times$ below that. At $N = 5$ the tree performs 18 force-term evaluations where direct summation performs 10, and builds a 7-node tree 20,000 times on top. Note also

that $\log_2 5 = 2.3$, so “ $O(N \log N)$ beats $O(N^2)$ ” compares $5 \times 2.3 \approx 12$ against $25/2 \approx 12$: **there is no asymptotic gap to exploit at this size**. Barnes–Hut is the *cheaper* tree code, so this is a strict lower bound on FMM’s constant factor. pyperformance itself ships `bm_barnes_hut` as a *separate* benchmark — upstream treats the tree code as a different workload, not an optimization of this one.

- **Struct-of-arrays loses (0.88×)**. Textbook advice, wrong here. In C, `x[i]` is a pointer add and the lanes vectorize; in CPython it is a `PyObject_GetItem` round trip, and the stock layout already hands the loop two *direct* velocity-list locals. SoA raises subscripts per step from 150 to 245. A data-layout optimization is only an optimization relative to a cost model, and the interpreter’s cost model is not the hardware’s.
- **NumPy loses (0.35×)**. At vector length 5–25 there is nothing to amortize the per-call dispatch against.

We also disproved a widely-repeated micro-optimization recorded in our own research notes: “stop destructuring positions, index `r1[0] - r2[0]` directly”. Measured against a semantic-no-op control, it is worth **1.005× — nothing**. The real win in that tier (1.12×) came from a different change: turning six subscript read-modify-writes into six plain stores.

3c. Raising N: where partial evaluation stops being practical

The stock benchmark hard-codes five bodies. Since the emitter is generic in `N`, `--bodies N` appends `N-5` further bodies on deterministic circular orbits — closed form, no RNG, semi-major axes from 40 AU outwards on golden-angle phases — and regenerates `advance()` for the larger system. The default is unchanged and checked structurally, not assumed: at `N = 5` the emitted source is still byte-identical to what it was before the flag existed.

This is worth doing because the technique has a cost that only shows up as `N` grows. Unrolling is $O(N^2)$ in **source size**: 10 pairs at `N = 5`, 4,950 at `N = 100`. The natural expectation is that the speedup collapses once the generated function outgrows the instruction cache.

It does not. Measured against a stock kernel running the identical system, at constant total pair-updates:

N	pairs	stock	generated	speedup	Rust
5	10	139.9 ms	99.4 ms	1.51×	23.4×
20	190	126.1 ms	85.9 ms	1.49×	22.6×
50	1,225	136.1 ms	87.6 ms	1.46×	25.7×
100	4,950	132.4 ms	94.2 ms	1.42×	25.2×
200	19,900	139.8 ms	94.1 ms	1.41×	26.9×
400	79,800	126.5 ms	98.8 ms	1.37×	22.2×

The win decays gently from $\sim 1.5\times$ to $1.37\times$ and never approaches 1.0 up to `N = 400`, while Rust holds $22\text{--}27\times$ flat — $O(N^2)$ in time but $O(1)$ in code size, which is exactly the asymmetry that motivates moving this kernel off the interpreter entirely. Output stays bit-identical to stock at every `N` tested.

What **does** become prohibitive is the bill partial evaluation moves to import time. `compile()` is linear in pairs, i.e. quadratic in `N`, at roughly $55\text{--}70\ \mu\text{s}$ per pair: 0.5 ms at `N = 5`, 1.3 s at `N = 200`, 9 s at `N = 500`. Since `pyperf` spawns ~ 20 worker processes that each re-import the module, an A/B at `N = 200` pays ~ 30 s of pure compilation. Peak compiler working set grows the same way — ~ 40 kB per emitted pair, 2.4 GB at `N = 400` — and `N  $\approx$  600` exhausts the machine. So the honest statement is not that unrolling stops being a **speedup**, but that it stops being **practical** around `N = 100–200`, because the cost it moves grows one order faster in `N` than the cost it removes.

The one real cliff, and the fix. An early sweep showed a sharp, reproducible step between `N = 30` and `N = 32` — $1.45\times$ dropping to $1.30\times$ — which no cache argument explains at 67 kB of bytecode. The cause is an interpreter detail: `LOAD_FAST/STORE_FAST` carry a one-byte operand, so local slot ≥ 256 needs an `EXTENDED_ARG` prefix. The emitter uses 8 locals per body plus temporaries, so `nlocals` crosses 256 at exactly `N = 31`.

N	nlocals	bytecodes / pair	EXTENDED_ARG
30	250	77.4	0.0%
31	258	85.2	9.4%
32	266	103.5	25.5%
200	1,610	114.3	36.3%

What made it hurt disproportionately was **ordering**. `co_varnames` is ordered by first binding, and `dx`, `dy`, `dz`, `mag`, `b1m`, `b2m` were bound inside the loop — last — so the six hottest names in the function, touched ~ 20 times per pair, were precisely the ones pushed past the boundary. Emitting one dead-store line at the top moves them to slots 3–8. It changes no arithmetic and is emitted only when $8 \cdot \text{len}(\text{bodies}) + 10 > 255$, so the shipped `N = 5` source is untouched. Paired A/B: **1.12× at `N = 32`, 1.11× at `N = 50`, 1.13× at `N = 200`**, with `N = 20` as a built-in control at $1.002\times$ because the fix is not emitted there. Note that 23% fewer bytecodes bought only 13% more speed — bytecode count is not the binding constraint at large `N`.

This is the kind of finding that only appears when a knob is swept rather than reasoned about, and it is a concrete instance of the course’s framing that the interpreter, not the physics, is the machine being programmed here.

4. Performance comparison

Stock and optimized were run back-to-back in the same session on the VM with `pyperformance run --rigorous`, the optimized build through a custom `--manifest` that keeps the benchmark name identical so `pyperf compare_to` matches by name. Significance is Student’s two-tailed t-test at 95% confidence.

Configuration	Mean ± std dev	Speedup	Correctness
Stock pyperformance 1.14.0	231 ms ± 3 ms	1.00×	reference
Optimized (shipped, pure Python)	141 ms ± 4 ms	1.64×	bit-identical state + energy

A **39.0% reduction in runtime** against a requirement of 7% — **5.6× the bar** — and the difference is reported significant. On CPython 3.12 the same change measures ~1.5× rather than 1.64×: 3.11+ adaptive specialization already recovers part of the subscript overhead we remove, so the win is *larger* on the 3.10 the course targets. Re-running on a newer interpreter should be expected to give a smaller number, and that is not a regression.

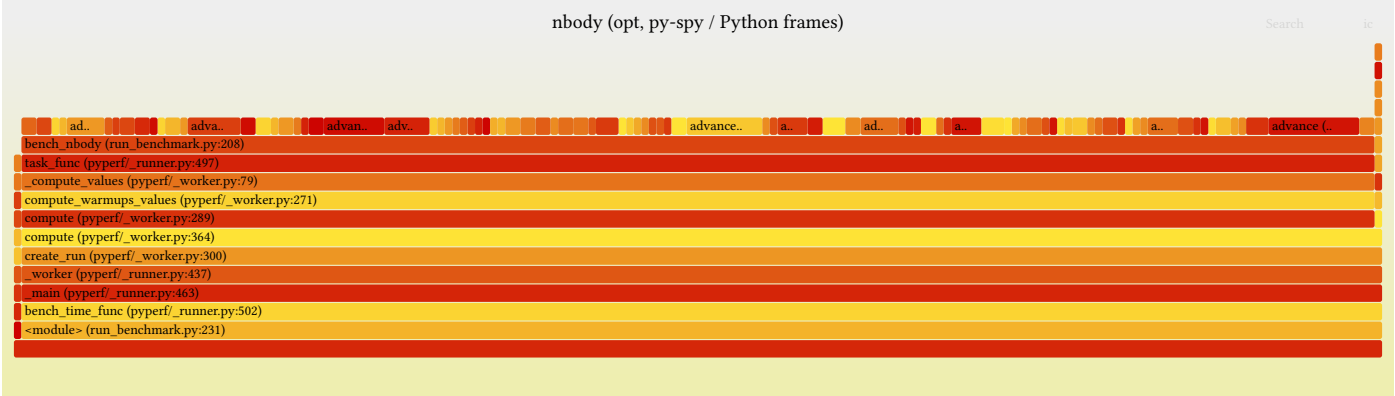


Figure 4: **After, Python frames.** Same sampling settings and workload flags as the “before” pair, so the comparison is direct — and the same 2.7% of import-time samples excluded, taking it from 41 rows to 16.

A native tier, for scale

We also built the kernel as a Rust/PyO3 extension (`rust/nbody/`): state resident in a `#[pyclass] System` holding `Vec<Vec3>`, one FFI crossing per `advance()`. On the VM it runs the 20,000-step integration in **9.48 ms against 228.49 ms — 24.1× on the kernel** — and its output is *also* bit-for-bit identical, because `f64::powf(-1.5)` lowers to the same glibc `pow` CPython calls and Rust neither contracts to FMA nor reassociates floats by default. The FFI crossing costs ~224 ns, or $2e-5$ of one `advance()` call, which is why the coarse boundary is the right one.

It is now wired into the benchmark, behind an explicit switch. The original objection to an optional import — that a silent fallback makes the measured configuration ambiguous — is a real one, so the import is not silent. `HWSW_BACKEND` pins the back end to `auto`, `python` or `native`; `native` fails loudly rather than degrading if the extension is missing; and the back end that actually ran is written into the `pyperf` metadata of every result JSON, so a measurement cannot claim to be something it was not.

What ships is the shape Lecture 5 (“Accelerator Design Patterns”) prescribes for an accelerator’s software interface. The user’s command line does not change (Rule 1). Every native line is confined to a separate crate behind one call (Rule 2). A host without the wheel runs the Python path instead of breaking (Rule 3) — checked on a machine with no wheel installed, where the module imports, binds `None`, and still produces bit-identical output with the same energy. The stock-versus- optimized comparison in the table above remains the pure-Python A/B, because that is the software-optimization result the project asks for; the native number is reported as the tier on top of it, never folded into it.

Measured end to end on the course VM, `pyperf --rigorous` (41 processes a side), the two back ends of the **same file** on the **same** interpreter, differing only in `HWSW_BACKEND`:

Whole benchmark, course VM	Python	native	speedup
nbody, 20,000 steps	141 ms	9.49 ms	14.86×

The whole-benchmark figure (14.86×) is smaller than the kernel figure (24.1×) and that gap is the honest part: `report_energy()` is still the stock $O(N^2)$ Python loop and is called twice per iteration, so it is un-accelerated on both sides and drags the ratio down. Quoting the kernel number alone would hide it.

The boundary is coarse on purpose, and for the reason the TPU case study gives for preloading weights before streaming inputs: crossing the interface costs more than the work when the payload is small. The kernel owns the state for the whole run and `advance()` is a doorbell, so there is one crossing per call rather than 20,000. Marshalling 6N Python floats across the FFI every step

would have cost more than the interpreter overhead being removed. As §5 argues, that same object is the accelerator’s register file, which is why the crate doubles as the behavioural specification.

5. Hardware acceleration proposal

§2 showed the arithmetic is cheap and the interpretation expensive; §4 showed that even after removing the interpreter entirely, the remaining cost concentrates in one operation: $d^{(-1.5)}$, a `libm pow` call per pair per step. That is the case for fixed-function hardware.

Precedent. The GRAPE machines (GRAPE-1...8, University of Tokyo, Makino et al.) were real silicon built for exactly this force law and won Gordon Bell prizes; GRAPE-8 reports 20.5 Gflops/W. The designs were never released as open RTL — the published record is papers, not source — so the design below is our own, informed by theirs.

Datapath (per pair, fully pipelined): 3 subtractions for $dx, dy, dz \rightarrow$ 3 multiplies + 2 adds for $d^2 \rightarrow d^{(-1.5)}$ via an `rsqrt` ROM seed plus two Newton–Raphson iterations $\rightarrow$ two mass multiplies $\rightarrow$ six velocity accumulate-FMAs. A second short pipeline integrates the five positions once per step. **Control:** an FSM sequences all ten pairs then the position update; the only cross-step dependency is the state itself, so the pair loop pipelines freely while steps stay serial.

Inputs / outputs and operating frequency. FP64 datapath (FP32 carried as an area design point). MMIO register map: body-state load window ($5 \times 7 \times 64$ bit), DT (64), NSTEPS (32), START doorbell, STATUS/IRQ, and a read-back window. Target **100 MHz** on FPGA; at ~20–30 cycles per step that is ~500k cycles $\approx$ **5 ms**, against 231 ms in Python and 9.5 ms in Rust.

HW/SW interface. The key decision is the NSTEPS register: software writes `dt` and `n = 20000` once and rings the doorbell *once*, collapsing 20,000 interpreter-driven iterations into a single MMIO transaction. A thin character driver plus a `ctypes/PyO3` wrapper exposes it to Python. **And the software work already specified it:** the Rust `#[pyclass]` from §4 is the register file, `advance(dt, n)` is the doorbell write, and `state()` is the read-back window. One boundary, three implementations.

Justification. `advance()` is ~95% of runtime, so Amdahl is unusually kind — offloading it bounds the whole benchmark rather than a slice of it. Assumptions: state resident on-device across steps (no per-step DMA), one pair per pipeline pass, and a fully pipelined `rsqrt` unit.

Performance / area / frequency / power trade-offs. Three knobs. *Replication:* one pipeline reused across the ten pairs (small area, ~10× latency) versus ten replicated pipelines (~10× area, near-single-step latency). *Precision:* FP64 $\rightarrow$ FP32 roughly halves multiplier area and shortens the critical path, buying frequency at a precision cost that is directly measurable against the benchmark’s own energy oracle — an experiment our bit-identical software baseline makes meaningful. *rsqrt:* ROM seed width against Newton–Raphson iteration count trades table area for latency; this is the operation that still dominates even the compiled Rust kernel, so it is where fixed-function hardware earns its win over software. On power, the GRAPE lineage is the evidence that a fixed-function gravity pipeline reaches roughly an order of magnitude better Gflops/W than a general-purpose core on this kernel.

6. Conclusion

The benchmark looks like a physics problem and behaves like an interpreter problem. About a sixth of stock runtime goes into turning the constant `0` into an array index, which is why the change that paid was not a better algorithm but the removal of 146 of 150 list subscripts per step: **1.64× on the course VM, 39% less runtime against a 7% requirement, with bit-for-bit identical output.** The Rust kernel takes the same idea to its conclusion at 24.1×, still bit-identical, and in doing so exposes the one remaining arithmetic bottleneck that only hardware fixes.

The negative results were worth more than the win. Barnes–Hut — and by extension FMM — is **3.4× slower** at $N = 5$ with a crossover near $N \approx 350$; struct-of-arrays, the standard data-layout advice, *loses* in an interpreter; NumPy loses at this vector length; and a widely-repeated micro-optimization is worth statistically nothing. Each cost less to establish than the effort it redirected. The same discipline paid a second time in hardware: the measured 95% share of `advance()` is what makes a step-count register worth building, and the measured residual cost of `pow` is what puts the `rsqrt` unit at the centre of the design.

Appendix: how the flame graphs were trimmed

A flame graph is exactly as tall as the single deepest stack in the profile, however rare that stack is, and ours were pathological that way. Left alone, the `perf` capture of stock `nbody` ran to 178 rows and took more than a page on its own. Three reductions, applied by `tools/trim_folded.py` and recorded in `tools/vm_remake_flames.sh`, bring them down. They differ in how much they cost, and the distinction matters more than the sizes do.

Dropping import-time samples (py-spy figures). `py-spy` starts sampling at process start, so it catches CPython importing `re`, `enum` and `collections` before the benchmark loop is entered. Those stacks are deep — `_find_and_load` $\rightarrow$ `_load_unlocked` $\rightarrow$ `exec_module` $\rightarrow$ `re._compile` — and there are only a handful of them, so they set the height while contributing nothing. In the optimized `py-spy` profile they are 2.7% of samples and take the figure from 41 rows to 16. This is not a truncation; it excludes a **phase** the figure was never meant to show.

Eliding the common prefix (perf figures). The C-level captures begin with interpreter start-up and the `pyperf` harness — 97 frames present in $\geq 95\%$ of samples. A frame in essentially every sample is constant context: it carries no information but costs a row. Every surviving frame keeps its exact sample count, so **no width in the plot changes**; only the y-origin moves. The threshold is 95%

rather than 100% because roughly 1.7% of DWARF unwinds are partial and start mid-interpreter, and at 100% those few fragments block the trim entirely (178 rows became 168 — effectively nothing).

Capping depth (perf figures). Even without the prefix, a thin tower of deep stacks kept the figure near a full page, so stacks are capped at the shallowest depth leaving 95% of samples whole. *This one genuinely discards detail:* the tips of 4.5% of samples are merged into their ancestors. It is the only one of the three that loses information, which is why each affected caption states the cut depth and the truncated share, and why the uncut graph is committed beside it as `*_full.svg`.

None of the three rescales anything, and no number quoted anywhere in this report is derived from a trimmed graph — the timings come from `pyperf`, and the self-time percentages from flat `perf report` output and `cProfile`, both taken on untrimmed data.